

Proyecto: Black & White Filesystem

Leister Alvarado (2017021458)

Elías Alvarado (2025800722)

Maestría en Ciencias de la Computación

Sistemas Operativos Avanzados

I. INTRODUCCIÓN

En relación al diseño y la implementación de los sistemas de archivos, en un aspecto más histórico ha sido una tarea reservada para el kernel space de los sistemas operativos. Por lo tanto, responde a la necesidad de realizar la gestión del hardware de almacenamiento considerando la menor latencia posible y con privilegios elevados, pero esta visión tiene riesgos de forma significativa porque si se presenta un error en el código del sistema de archivos puede comprometer lo que es la estabilidad de todo el sistema operativo, generando fallos que son críticos o en la pérdida de los datos [1].

Entonces como parte de soluciones para reducir esta complejidad y fomentar la innovación, fue donde surge la arquitectura de FUSE(Filesystem in Userspace). El FUSE actúa como una interfaz que conecta de una forma para dar a los desarrolladores una forma de implementar los sistemas de archivos completos que se ejecutan como procesos de usuario no privilegiado. Con respecto al módulo del kernel de FUSE intercepta las llamadas al sistema que serán las funciones presentadas más adelante y realiza una re dirección al programa de usuario, donde define cómo se almacenan y recuperan los datos de forma real [2]. Al tener este tipo de abstracción ha permitido la creación de sistemas de archivos no convencionales que no dependen de dispositivos que son bloques físicos tradicionales, sino que por ejemplo pueden utilizar redes, memoria RAM, o como en el caso de este proyecto un sistema de archivos black and white.

Además para el presente proyecto sobre Black and White Filesystem pues se propone un cambio en la paradigma de la persistencia de los datos, por lo que primero se trabaja con bloques físicos con cierto i-nodo como nuestro sistema de archivos y luego se realiza una transformación de todo a un BWFS que utiliza imágenes en blanco y negro como un tipo de soporte físico de almacenamiento, más adelante se explicará mejor sobre esto.

Para nuestro modelo, pues el concepto de bloque de disco se reinterpreta como una región de píxeles dentro de una imagen, entonces la información binaria se codifica de una forma visual, donde contemplando todos los píxeles representan pues los bits de datos y metadatos. El sistema tiene la posibilidad de realizar la gestión de estas imágenes que se limitan con una dimensión de 1000x1000 píxeles por cada bloque, lo anterior con el objetivo de realizar la construcción de una estructura que sea lógica y coherente para alojar archivos de cualquier tipo y tamaño.

La implementación de BWFS pues plantea ciertos desafíos

de forma significativa con temas de traducción, concurrencia y sistemas distribuidos, es capaz de mapear operaciones de alto nivel POSIX a manipulaciones gráficas de bajo nivel, por lo que se conforma de tres componentes que son fundamentales:

- **mkfs.bwfs**: Corresponde a un binario que se encarga de formatear el medio, donde establece las estructuras de indexación iniciales como i-nodos y define los parámetros de la imágenes.
- **mount.bwfs**: Es nuestro proceso daemon que monta nuestro sistema de archivos en el árbol de directorios del GNU/Linux, interpretando de forma real las solicitudes del usuario.
- **bwfs_export_images**: En forma general funciona como un transformador de nuestros bloques del file system que para cada i-nodo representa un archivo a lo que es un BWFS en un directorio nuevo con cada archivos convertido.

Otra de las funcionalidades que presenta nuestro sistema de archivos es la implementación de naturaleza distribuida, entonces el BWFS no opera únicamente de manera local, tiene mecanismos de comunicación TCP/IP para gestionar la persistencia de el acceso a los datos con una representación de master y slaves, desacoplando la lógica del sistema de archivos de la ubicación física de las imágenes.

II. AMBIENTE DE DESARROLLO

Este proyecto está fundamentado en 2 pilares; el primero de ellos se basa en el envío de paquetes de datos por medio del protocolo **TCP/IP** y el segundo en la creación de un sistema de archivos distribuidos. Para la realización de las pruebas, se buscó un ambiente de trabajo lo más similar a lo que se esperaría en dispositivos físicos reales, por este motivo, durante su construcción se usó como criterio decisivo un ambiente que permita asemejar el envío de paquetes remotos y la sincronización del sistema ante cualquier evento generado por el usuario. A continuación se detallan las tecnologías utilizadas:

- **C**: Lenguaje de programación por defecto que proporciona acceso a la librería FUSE utilizada para la construcción del **BWFS**.
- **CMakeLists.txt**: Herramienta utilizada para la compilación de binarios. Su resultado se genera en la carpeta `./build`.
- **Github**: Plataforma donde se publica y gestiona el código desarrollado.
- **Docker**: Permite la contenerización de sistemas para las pruebas de archivos distribuidos.

III. ESTRUCTURAS DE DATOS USADAS Y FUNCIONES

Este proyecto está compuesto por diferentes funciones y estructuras que fueron implementadas siguiendo el modelo establecido por FUSE. Para un mayor orden se explicará de forma ordenada por código:

- **bwfs.c**: En este código se implementa un sistema de archivos basado en FUSE3 que tenemos un modelo simplificado de tipo un archivo por nodo, entonces cada archivo lógico que el usuario ve en el directorio montado pues corresponde de forma física a un archivo real en el disco dentro del directorio de respaldo de bwfs storage, además tenemos metadata donde almacenamos magic number, ruta absoluta del storage, tamaño máximo permitido por archivo y la tabla completa de los i-nodos. Las estructuras principales son las siguientes:

1. **inode_t**: Este representa un archivo o directorio dentro del FS, entonces contiene `used` que es si el inodo esta ocupado, el `name` para el nombre del archivo, `mode` para permisos, `uid` y el `gid` para propietario y grupos, `size` es el tamaño y `atime`, `mtime` y `ctime` para timestamps donde básicamente equivale lo anterior a un `struct_stat` para cada archivo.
2. **bwfs_disk_t**: Representa la metadata persistente que esta almacenada dentro del `bwfs_metadata.bin` donde incluye el magic que es una constante, el storage path que es la ruta donde se almacena el `.dat`, el max block byte es el tamaño de cada archivo y los `inodes[MAXFILES]` que es la tabla completa de inodos.
3. **bwfs_t**: Es el estado en memoria de RAM del sistema de archivos una vez montado, se carga desde el `bwfs_metadata.bin` al iniciar y se sincroniza a disco por medio del `save_metadata()`.

Ahora continuamos con la inicialización del sistema de archivos, donde es el `init` que FUSE llama para montar de forma automática el FS. Las tareas que tiene son cargar la metadata desde el bin, en caso de no existir la crea, normaliza la ruta del storage, asegura que el storage exista y configura los parámetros globales, donde al final se tiene como resultado en memoria la tabla de i-nodos y el estado correcto a operar. Continuando sigue las funciones de gestión de metadata:

1. **load_metadata** que realiza la lectura del archivo bin y carga lo que es el magic number, ruta del storage, tamaño máximo del bloque, inodos persistentes y en caso que algo falle pues el FS arranca con valores por defecto.
2. **save metadata**, realiza el almacenamiento del estado actual del FS en el disco entonces lo que hace es un relleno de un `dist t`, donde luego copia la tabla de i-nodos y el storage path con sus parámetros, al final escribe todo en un único `fwrite`. Esto permite que el FS recuerde de forma permanente el nombre del archivo, tamaños, permisos, propiedades y contenido, incluso después de desmontarlo.

Para el caso del manejo del directorio de almacenamiento, iniciando con el `normalize storage path` pues convierte rutas relativas en absolutas, luego el `ensure storage` verifica o crea un directorio físico en el storage. Continuando para el caso del manejo de nombre e inodos, primero se tiene el `basename from path` que extrae el nombre del archivo desde el directorio del montaje, también el `find inode by name` busca un archivo en la tabla de inodos donde retorna el índice del i-nodo o -1 en caso que no exista. Y por último de esta parte el `allocate inode` que describe el proceso completo de la creación de un archivo, inicia con la búsqueda de un i-nodo libre, luego lo marca como usado, copia nombre sus permisos y timestamps para luego crear el archivo físico `.dat` y guarda la metadata persistente, es una pieza útil para el `create`, `touch` y demás.

La otra parte del código del bwfs y una de las más importantes es la implementación de las funciones FUSE:

1. **bwfs_getattr()**: Realiza un retorno del `stat` de un cierto archivo, el tamaño, permisos, propietario, tipo, por ejemplo los comandos `ls`, `stat`.
2. **bwfs_readdir()**: Muestra la lista de archivos dentro del /, entonces se añade el recorrido de i-nodos `used` y retorno de nombres.
3. **bwfs_create()**: Simplemente llama un `allocate_inode` en nuestro caso entonces por ejemplo en comandos de `touch`, `nano`, etc pueden funcionar pero sin editar aún.
4. **bwfs_open()**: Realiza una validación simple donde verifica si existe el archivo para poder abrirlo o no.
5. **bwfs_read()**: Lo que busca es realiza una lectura del contenido de un `.dat` entonces el flujo primero busca un i-nodo, luego verifica el `offset`, abre el archivo, funcionalidad `lseek`, el `read` hacia el buffer de FUSE y actualiza `atime`, donde hace un retorno de bytes leídos.
6. **bwfs_write()**: Realiza una escritura en un `.data`, el flujo es buscar el i-nodo, valida que no se exceda del tamaño establecido, abre el archivo físico, funcionalidad de `lseek`, luego realiza un `write`, actualiza el tamaño, guarda la metadata nueva y replica el slave de `write rename file`.
7. **bwfs_unlink()**: Es un equivalente al `rm` donde busca eliminar solo archivos, entonces busca primero un inodo, luego borra el archivo físico, luego marca el i-nodo como libre y guarda la metadata como actualización.
8. **bwfs_rename()**: Solamente busca cambiar el nombre dentro del i-nodo. Por ejemplo utilizando el comando `mv`.
9. **bwfs_access()**: En general, realiza una validación de permisos de POSIX según sea el owner, grupo, otros y el uid real.
10. **bwfs_chmod()**: Es un función extra que decidimos implementar para realizar la actualización de permisos almacenados por inodo que nos parece una buena funcionalidad.

11. `bwfs_utimens()`: Otra función extra, que nos ayudó en solucionar ciertos bugs entonces realiza una actualización del tiempo de acceso y el tiempo en la modificación.
 12. `bwfs_statfs()`: Realiza el reporte del estado del sistema de archivos, donde muestra los bloques totales, los libres, el tamaño del bloque por ejemplo los comandos `df -h mnt/` y `stat -f mnt/` que fueron útiles porque verificamos el tamaño de cada bloque que cumple con lo solicitado.
 13. `bwfs_fsync()`: Realiza un mandato en escritura completa del archivo físico.
 14. `bwfs_flush()`: Esta función se llama cuando un archivo se cierra.
 15. `bwfs_mkdir()`: Básicamente permite crear nuevos directorios dentro del FS montado, cabe destacar que otras dos funciones opcionales por implementar no fueron consideradas porque teníamos que heredar y modificar muchos aspectos de `i` nodos y demás por lo que no se consideraron.
 16. `bwfs_lseek()`: Para nuestra versión de FUSE3 investigando no lo implementamos, ya que se encuentra en la librería, cómo explicamos en el código los offsets ahora con manejados por FUSE de una forma automática.
- `mkfs_bwfs.c`: Para este programa de `mkfs` tiene la función de crear y formatear un nuevo sistema de archivos, entonces inicializa de forma correcta toda la estructura base del FS, garantizando que realmente se creen; el archivo de los metadata persistente del bin, la tabla inicial de `i`-nodos, el directorio de almacenamiento físico los 1024 bloques físico que representan archivos reales en el almacenamiento `.dat`. Utiliza una estructura parecida explicada antes de los `inode t` para la estructura del `i`-nodo lógico, los `i`-nodos son la tabla de archivos del FS, son elementos que BWFS usa para saber qué existe dentro del FS y el programa inicializa 1024 inodos que están vacíos representando el FS que recién fue creado. Luego el `bwfs_t` que la metadata global para el FS pues representa el estado completo del FS al momento de formatearlo entonces al final esta estructura es exactamente lo que se escribe dentro del archivo del bin entonces permite recordar aspectos importantes como su configuración, cuántos archivos existen, sus nombres respectivos y qué bloques están ocupados. Ahora se presentarán las funciones principales del código `mkfs`:
 1. `read config`, representa la lectura del archivo `config.ini` con el objetivo de leer los parámetros del FS que están definidos por el usuario es decir el storage path y el max block bytes, lo anterior lo que hace es abrir este `config.ini` en el modo de lectura, realiza el recorrido del archivo buscando líneas reconocibles con `sscanf`, luego extrae los valores y los almacena en variables temporales y al final en caso que el parámetro no exista pues se usa su valor por defecto. Permite que nuestro FS sea configurable y pueda almacenar sus bloques en cualquier directorio seleccionado por el usuario.
 2. `make absolute path`, es una normalización de rutas donde como objetivo busca garantizar que la ruta storage sea absoluta, válida, donde al final asegura que el storage siempre sea un ruta consistente, sin importar de cómo el usuario lo ingrese.
 3. Inicialización de la metadata, este paso en forma general no se realiza exactamente cómo se explica en el código anterior, el FS no será reconocido por el montaje del FUSE.
 4. Creación del directorio storage, verifica si este existe en caso que no pues lo crea con permisos 0755, es importante porque este directorio almacenará todos los bloques físicos.
 5. Escritura del archivo bin, donde busca guardar la estructura completa del `bwfs t` en un solo archivo binario, esto representa el superbloque del BWFS, sin este archivo pues el FS no tiene una identidad.
 6. Creación de 1024 bloques físicos, nuestro FS usa un modelo 1 bloque físico por archivo, entonces estos archivos representan los bloques donde se almacenará el contenido real.
 - `mount_bwfs.c`: Este programa es auxiliar donde cuya función es preparar y ejecutar el sistema de archivos de nuestro sistema de archivos, lo anterior por medio de la invocación del binario `bwfs` del daemon del FUSE. Entonces este programa se encarga: validar los parámetros de entrada, comprobar que el sistema FS fue inicializado con `mkfs`, ubica de forma correcta los directorios relevantes, crear el directorio de montaje si no existe, resuelve rutas absolutas, y ejecuta el daemon del FUSE real.
- Otro programa importante que se desarrolla es el `bwfs_export_images.c` está diseñado para transformar nuestros archivos del sistema de archivos binarios en imágenes PBM, entonces la función encargada de cargar archivos en memoria es denominada `load_file`, su objetivo es abrir un archivo en modo binario, realizar el cálculo de su tamaño, una reserva en memoria suficiente y copiar todo el contenido en un buffer. De forma tal que cuando un archivo es leído completamente y manipulado en memoria pues el resultado es un bloque de bytes junto con su tamaño y estos ayuda que otras funciones logren trabajar con este sin tener problemas en detalles de lectura. Luego una vez que se tiene un bloque de binarios, la función `buffer_to_pbm` realiza la conversión en una imagen en formato PBM, para lograr esto pues realiza el cálculo del tamaño mínimo de una imagen cuadrada que puede contener todos los bytes del buffer. Luego crea un archivo PBM con la cabecera correspondiente y recorre cada posición de la cuadrícula, realizando la asignación de un píxel blanco o negro según sea el valor del byte, se establece que si el byte es mayor o igual a 128 se interpreta como blanco y si es menor pues negro, de forma tal que los datos binarios se transforman en una representación visual que permite ver la información en forma de imagen y otra función importante es `ensure_dir` que asegura una existencia del directorio antes de guardar archivos con este, entonces si el directorio no se encuentra presente, lo

crea con permisos completos, pero en caso que falle la creación pues muestra un error y detiene la ejecución, y esto lo que busca es garantizar que el programa siempre tenga un lugar válido donde escribir las imágenes generadas. Finalmente en el main, muestra un mensaje de inicio y asegura que el directorio exista, luego carga el archivo que tenemos nuestra metadata y convierte todo lo que tenemos en nuestro bwfs_storage que son los bloques que cada uno representa un i-nodo conectado a un cierto archivo creado en el sistema de archivos en una imagen PBM. Parte del funcionamiento que establecimos fue que se realiza al final esta conversión para cumplir con el B&W file system.

El ambiente distribuido se realizó por medio de estructuras básicas para el envío y recepción de datos, donde se configuró un host master y un host esclavo. Ambos hacen uso del mismo software, con la diferencia de que el master tiene como propósito de informar a los demás de todos los eventos sucedidos en su ambiente mientras que los esclavos esclavos replican a su maestro. En el siguiente código se visualiza la estructura utilizada por cada paquete cuando es emitida a su destino.

```
1 typedef struct {
2     uint16_t magic;
3     uint8_t  version;
4     uint8_t  type;
5     uint32_t length;
6     uint32_t sequence;
7 } message_header_t;
```

Cada atributo tiene un propósito que lo convierte en esencial. Magic define el número único del protocolo, version indica la versión que se está usando del sistema. Type, por otro lado, identifica el tipo de operación que está utilizando en el usuario durante el envío y recepción de datos.

Cada cliente y servidor envían sus paquetes por medio de la función send_message usando como base el protocolo creado. Su contenido está determinado en el atributo type indistintamente del origen, por lo que la conexión se crea una vez que ambas partes hayan realizado un *handshake* y esté de acuerdo en la transmisión de información.

IV. INSTRUCCIONES PARA EJECUTAR EL PROGRAMA

Las instrucciones para ejecutar el programa se muestran en los siguientes pasos, importante mencionar que debe cumplirse cada paso, sino podría dar algún error. Además para el paso 3 donde se monta nuestro sistema de archivos pues queda ejecutándose en segundo plano por lo que para el paso 4 que se interactúa con el sistema de archivos se debe abrir otra terminal dentro de la IDE que se encuentre utilizando o terminal dependiendo del lugar de ejecución.

Paso 1: Compilar el código fuente.

```
cmake -B build -S . && cmake --build build
```

Paso 2: Crear el sistema de archivos.

```
./build/mkfs.bwfs -c config.ini
```

Paso 3: Montar el sistema de archivos con la estructura obtenida en el paso anterior.

```
./mount.bwfs -c config.ini mnt
```

Paso 4: Interactuar con el BWFS, en la otra terminal entrar al directorio donde se esté el mnt creado y puede realizarse todas las funciones implementadas.

```
echo "Hello, world" > mnt/output.txt
```

Paso 5: Ejecutar el bwfs_export_images, donde exporta a imágenes lo que tenemos de archivos. En nuestra implementación se realiza hasta el final para exportar literal lo que tengamos en nuestro sistema de archivos actual.

```
bwfs_export_images
```

Paso 6: Abrir como imagen, entonces ingresar al directorio nuevo storage_images que tiene los mismos índices que conocemos de nuestro storage donde almacena los archivos de nuestro sistema de archivos.

```
eog storage_image/ block_XXXX.pbm
```

V. ACTIVIDADES REALIZADAS

BITÁCORA DE TAREAS

Fecha	Tarea	Horas	Notas
12/11/2025	Implementación Funciones BWFS, mount y mkfs	5	Investigación sobre librerías
13/11/2025	Implementación Funciones BWFS, mount y mkfs	4	Desarrollo de los tres códigos
14/11/2025	Implementación Funciones BWFS, mount y mkfs	6	Desarrollo de los tres códigos
15/11/2025	Implementación Funciones BWFS, mount y mkfs	4	Desarrollo de los tres códigos
22/11/2025	Investigar el protocolo TCP/IP en C	3	Investigación y diseño de las computadoras
23/11/2025	Implementar el cliente / servidor	7	Crear el código que permite enviar y recibir mensajes dentro de un mismo host
30/11/2025	Configuración de Docker	2	Crear el dockerfile para ejecutar en un contenedor la solución desarrollada
28/11/2025	Creación de convertidor a BWFS	7	Se piensa en modificaciones en los códigos pero podría generar errores por lo que decidimos hacer un código que convierta todo a imágenes B&W

Sobresalen reuniones cada dos o cuatro días aproximadamente como checkpoint sobre que llevamos, ayuda, errores, planificación por solucionar juntos.

VI. AUTOEVALUACIÓN

VI-A. Evaluación

Cuadro II: Desglose de la Evaluación del Proyecto

Componente	Porcentaje (%)
BWFS Total	55 %
• mkfs.bwfs	14 %
• mount.bwfs	15 %
• Funciones de la biblioteca	26 %
◦ getattr	
◦ create	
◦ open	
◦ read	
◦ write	
◦ rename	
◦ mkdir	
◦ readdir (opcional)	
◦ opendir (opcional)	
◦ rmdir (opcional)	
◦ statfs	
◦ fsync	
◦ access	
◦ unlink	
◦ flush	
◦ lseek	
Documentación	20 %
Persistencia en Disco	25 %

VI-A1. Estado del entregable: El proyecto se desarrolló en su totalidad, completando sus partes fundamentales solicitadas, que corresponden a un sistema de archivos con características especiales de interpretación como imágenes en blanco y negro equivalentes a los i-nodos y al súper bloque. Adicionalmente, la solución contiene una implementación distribuida de archivos que replica los eventos sucedidos en el host maestro hacia sus esclavos cumpliendo de manera exitosa el requerimiento de un ambiente distribuido, además cabe destacar las funciones implementadas fueron de gran ayuda para realizar verificaciones tanto de diseño y modularidad del programa.

Adicionalmente, se creó un **video demostrativo de la solución** que ejemplifica cada funcionalidad solicitada. Al inicio se muestran las diferentes operaciones soportadas como resultado de la integración de la librería FUSE y finalmente, se concluye con un caso de uso de beneficio al tener un sistema de archivos distribuido. El en el siguiente enlace puede encontrar el video demostrativo: Demo BWFS.

VI-A2. Evidencia de Trabajo: Trabajamos en conjunto por medio de reuniones sincrónicas virtuales donde realizamos sesiones de pair-programming y utilizamos Github como herramienta de trabajo colaborativo, que nos facilitó el seguimiento y revisión de los cambios realizados durante el desarrollo del proyecto.

El repositorio tiene un acceso privado como mecanismo de protección anti-plagio y está hospedado en el siguiente enlace: soa-black-and-white-fs.

dentro de GNU/Linux, el realizar la replicación de comportamientos como el montaje del sistema, la separación de metadata y los bloques físicos con la validación del entorno pues refuerza el aspecto en desarrollar un código escalable y predecible, donde luego se tenga un ambiente de sistema más distribuido.

A nivel conceptual, es sencillo de entender cómo funciona el envío de paquetes de red; al realizar la implementación de un concepto a una solución práctica el proceso es bastante enriquecedor. Comprender como se computa un dato complejo a una versión primitiva que pueda entender el servidor para procesarlos y enviar los datos brinda una mejor perspectiva de la computación y del uso cotidiano que se le da a un protocolo de red como lo es TCP/IP.

REFERENCIAS

- [1] M.Szeredi, "Filesystem in Userspace", en Proceedings of the Linux Symposium, vol. 1, pp. 279-282, 2005. Disponible: <https://www.kernel.org/doc/html/next/filesystems/fuse.html>
- [2] R.Patel, "File System Concepts: A Deeper Understanding", Medium, Feb.2025. Diponible: <https://medium.com/@ravipatel.it/file-system-concepts-a-deeper-understanding-c879697b38c8>

VII. LECCIONES APRENDIDAS

En el proceso del desarrollo del BWFS permitió que comprendiera de una forma mas profunda sobre cómo funciona un sistema de archivos desde las bases, dando como desafío en enfrentar conceptos que pueden estar ocultos detrás de herramientas de alto nivel. El diseño de estructuras como la tabla de i-nodos, la metadata global, los parámetros del sistema y el manejo de múltiples bloques físico dan como refuerzo la importancia del orden, coherencia en los datos y una validación que fue rigurosa. Una de los mayores puntos de aprendizaje es del que un sistema de archivos no es simplemente guardar datos, sino de una estructura robusta de cómo se representan, organizan y recuperan esos datos en una forma más segura y lo más consistente posible.

Otra lección fue la interacción más directa con el sistema operativo y el proyecto en forma especial al implementar las funciones, que establece el cómo se gestionan las rutas, permisos, procesos y archivos